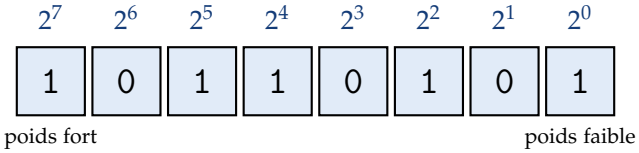
Université de Sousse
Institut Supérieur des Sciences Appliquées et de Technologie de Sousse (ISSAT Sousse)

Circuits Numériques

Chapitre : Systèmes de numération

Bases – Conversions – Arithmétique binaire – Nombres signés – Codes



$$(10110101)_2 = 128 + 32 + 16 + 4 + 1 = (181)_{10} = (265)_8 = (B5)_{16}$$

Licence 1ère année – Électronique, Électrotechnique et Automatique (EEA)

Support de cours avec exemples et exercices corrigés

Année universitaire 2026 – 2027

Table des matières

Introduction et objectifs **2**

1 Généralités sur les systèmes de numération **2**

1.1 Notion de base et numération de position . . . 2

1.2 Bases usuelles . . . 3

1.3 Vocabulaire et capacité de représentation . . . 3

2 Conversions entre bases **4**

2.1 De la base b vers le décimal . . . 4

2.2 Du décimal vers la base b . . . 4

2.3 Conversions entre bases 2, 8 et 16 : regroupement de bits . . . 5

2.4 Conversion entre deux bases quelconques . . . 6

3 Arithmétique binaire **6**

3.1 Addition . . . 6

3.2 Soustraction . . . 7

3.3 Multiplication et division . . . 7

4 Représentation des entiers relatifs **8**

4.1 Signe et valeur absolue (SVA) . . . 8

4.2 Complément à 1 (CA1, ou complément restreint) . . . 8

4.3 Complément à 2 (CA2, ou complément vrai) . . . 8

4.4 Comparaison des trois conventions . . . 8

4.5 Addition et soustraction en complément à 2 . . . 10

4.6 Dépassement de capacité (overflow) . . . 10

5 Codes binaires **11**

5.1 Le code DCB (BCD) . . . 11

5.2 Autres codes décimaux : Aiken et excès 3 . . . 11

5.3 Le code Gray (binaire réfléchi) . . . 11

5.4 Le code ASCII et Unicode . . . 12

5.5 Codes détecteurs et correcteurs d'erreurs . . . 13

6 Représentation des nombres réels **14**

6.1 Virgule fixe . . . 14

6.2 Virgule flottante : la norme IEEE 754 . . . 14

7 Synthèse du chapitre **16**

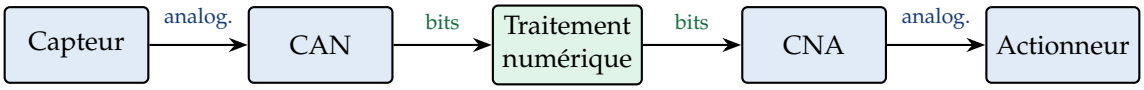
8 Exercices corrigés **17**

9 Exercices proposés **18**

Introduction et objectifs

Les systèmes électroniques modernes (microcontrôleurs, processeurs, circuits programmables, chaînes de mesure, réseaux de communication) traitent l'information sous forme **numérique**. Avant d'étudier les portes logiques et les circuits combinatoires et séquentiels, il faut savoir *comment un nombre, un caractère ou une valeur mesurée est représenté par des bits*. C'est l'objet de ce chapitre.

Signal analogique et signal numérique. Une grandeur **analogique** varie de façon continue (tension d'un capteur de température, signal audio). Une grandeur **numérique** ne prend qu'un nombre fini de valeurs discrètes, représentées par des nombres écrits avec des chiffres binaires (*bits*, de *binary digit*).



Pourquoi le binaire ? Un transistor utilisé en commutation possède deux états stables (bloqué ou saturé), ce qui correspond à deux niveaux de tension, par exemple 0 V pour le chiffre 0 et 3,3 V ou 5 V pour le chiffre 1 selon la technologie. Deux niveaux bien séparés résistent mieux au bruit que des valeurs continues, se copient et se stockent sans perte, et se traitent avec des circuits simples.

Objectifs du chapitre. À la fin de ce chapitre, l'étudiant doit être capable de :

1. écrire un nombre dans les bases 2, 8, 10 et 16 et passer d'une base à l'autre ;
2. effectuer les quatre opérations en binaire ;
3. représenter les entiers relatifs (signe et valeur absolue, complément à 1, complément à 2) et détecter un dépassement de capacité ;
4. utiliser les codes usuels (DCB, Aiken, excès 3, Gray, ASCII) et les codes détecteurs d'erreurs ;
5. représenter un nombre réel en virgule fixe et en virgule flottante (norme IEEE 754).

Prérequis. Puissances et logarithmes, division euclidienne, notation scientifique.

1. Généralités sur les systèmes de numération

1.1 Notion de base et numération de position

Définition

Un **système de numération** est défini par un ensemble de symboles, les **chiffres**, et par des règles d'écriture des nombres. Dans un système **positionnel** de **base** b (avec $b \geq 2$) :

- on dispose de b chiffres, de 0 à $b-1$ (au-delà de 9, on utilise des lettres : $A = 10$, $B = 11, \ldots$) ;
- la valeur d'un chiffre dépend de sa **position** : la position i a pour **poids** b^i.

Un nombre s'écrit $(a_{n-1} \ldots a_1 a_0 \,, a_{-1} \ldots a_{-m})_b$ et sa valeur est donnée par la **forme polynomiale** :

$$N = \sum_{i=-m}^{n-1} a_i \, b^i = a_{n-1} b^{n-1} + \cdots + a_1 b + a_0 + a_{-1} b^{-1} + \cdots + a_{-m} b^{-m}.$$

Le chiffre a_{n-1} est le chiffre de **poids fort** (MSD ; *MSB* en binaire), le chiffre a_{-m} celui de **poids faible** (LSD ; *LSB* en binaire).

1.2 Bases usuelles

Base	Nom	Chiffres	Utilisation
10	Décimal	0 à 9	usage courant
2	Binaire	0, 1	représentation interne des circuits numériques
8	Octal	0 à 7	écriture compacte (3 bits par chiffre), droits d'accès Unix
16	Hexadécimal	0 à 9, A à F	écriture compacte (4 bits par chiffre), adresses mémoire, couleurs

Déc.	Bin.	Oct.	Hex.	Déc.	Bin.	Oct.	Hex.
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Applications courantes. La couleur `#FF8000` est le triplet (rouge, vert, bleu) $= (\texttt{FF}, \texttt{80}, \texttt{00})_{16} = (255, 128, 0)$. Les droits Unix `rwxr-x---` valent `111 101 000` en binaire, soit $(750)_8$.

1.3 Vocabulaire et capacité de représentation

- **bit** : un chiffre binaire (0 ou 1) ; **quartet** : 4 bits ; **octet** : 8 bits ; **mot** : groupe de bits traité en une fois par un processeur (8, 16, 32 ou 64 bits) ;
- 1 ko $= 10^3$ octets, alors que 1 Kio $= 2^{10} = 1024$ octets ; de même 1 Mio $= 2^{20}$ octets et 1 Gio $= 2^{30}$ octets.

Capacité de représentation

Avec n chiffres en base b, on représente b^n nombres distincts, de 0 à $b^n - 1$.

- Nombre de bits pour représenter N valeurs distinctes : $n = \lceil \log_2 N \rceil$.
- Nombre de chiffres en base b d'un entier $N \geq 1$: $n = \lfloor \log_b N \rfloor + 1$.

n	0	1	2	3	4	5	6	7	8
2^n	1	2	4	8	16	32	64	128	256

n	9	10	11	12	13	14	15	16
2^n	512	1024	2048	4096	8192	16384	32768	65536

Exemple : nombre de bits

- Coder 1000 états distincts : $2^9 = 512 < 1000 \leq 1024 = 2^{10}$, donc $n = 10$ bits.
- L'entier 2026 : $2^{10} = 1024 \leq 2026 < 2048 = 2^{11}$, donc $\lfloor \log_2 2026 \rfloor + 1 = 11$ bits.
- **Application EEA** : un convertisseur analogique-numérique (CAN) de n bits et de tension pleine échelle V_{ref} a un pas de quantification $q = V_{ref}/2^n$. Pour $n = 12$ et $V_{ref} = 5$ V : $q = 5/4096 \approx 1{,}22$ mV.

2. Conversions entre bases

2.1 De la base b vers le décimal

On applique la forme polynomiale, ou la **méthode de Horner** qui évite de calculer les puissances : $a_3 b^3 + a_2 b^2 + a_1 b + a_0 = ((a_3 b + a_2) b + a_1) b + a_0$.

Exemple : conversions vers le décimal

- $(10110{,}101)_2 = 16 + 4 + 2 + \dfrac{1}{2} + \dfrac{1}{8} = 22{,}625$.
- $(3A7)_{16} = 3 \times 256 + 10 \times 16 + 7 = 935$. Avec Horner : $((3 \times 16) + 10) \times 16 + 7 = 58 \times 16 + 7 = 935$.
- $(752)_8 = 7 \times 64 + 5 \times 8 + 2 = 490$.

2.2 Du décimal vers la base b

Méthode : décimal $\rightarrow$ base b

Partie entière : divisions euclidiennes successives par b jusqu'à obtenir un quotient nul. Les **restes**, lus **de bas en haut** (le dernier reste est le poids fort), donnent l'écriture en base b.

Partie fractionnaire : multiplications successives de la partie fractionnaire par b. Les **parties entières** obtenues, lues **de haut en bas**, donnent les chiffres après la virgule. On s'arrête quand la partie fractionnaire devient nulle ou quand la précision voulue est atteinte.

Exemple : $(156)_{10}$ en binaire

Division	Quotient	Reste
$156 \div 2$	78	**0** (poids faible)
$78 \div 2$	39	**0**
$39 \div 2$	19	**1**
$19 \div 2$	9	**1**
$9 \div 2$	4	**1**
$4 \div 2$	2	**0**
$2 \div 2$	1	**0**
$1 \div 2$	0	**1** (poids fort)

En lisant les restes de bas en haut : $(156)_{10} = (\texttt{10011100})_2$. Vérification : $128 + 16 + 8 + 4 = 156$.

Exemple : $(0{,}6875)_{10}$ en binaire

Multiplication	Résultat	Partie entière
$0{,}6875 \times 2$	1,375	**1**
$0{,}375 \times 2$	0,75	**0**
$0{,}75 \times 2$	1,5	**1**
$0{,}5 \times 2$	1,0	**1**

La partie fractionnaire est nulle : $(0{,}6875)_{10} = (\texttt{0,1011})_2$. Vérification : $\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 0{,}6875$.

Remarque

Un nombre décimal à écriture finie n'a pas forcément une écriture binaire finie. Ainsi $(0{,}1)_{10} = (\texttt{0,0}\overline{\texttt{0011}})_2 = (0{,}000110011001100\ldots)_2$ est **périodique** : les multiplications successives ne s'arrêtent jamais. C'est la raison pour laquelle $0{,}1 + 0{,}2$ ne donne pas exactement 0,3 sur un ordinateur (voir la section 6).

Précision. Avec m bits après la virgule, la résolution est 2^{-m}. Pour obtenir une précision meilleure que 10^{-p}, il faut $m \geq p \log_2 10 \approx 3{,}32\, p$ bits. Par exemple, pour 3 décimales ($p = 3$) : $m = 10$ bits, car $2^{-10} \approx 0{,}00098 < 10^{-3}$.

Exemple : $(2026)_{10}$ en hexadécimal

$2026 = 16 \times 126 + \mathbf{10}$; $126 = 16 \times 7 + \mathbf{14}$; $7 = 16 \times 0 + \mathbf{7}$. D'où $(2026)_{10} = (\texttt{7EA})_{16}$. Vérification : $7 \times 256 + 14 \times 16 + 10 = 1792 + 224 + 10 = 2026$.

2.3 Conversions entre bases 2, 8 et 16 : regroupement de bits

Comme $8 = 2^3$ et $16 = 2^4$, un chiffre octal correspond exactement à **3 bits** et un chiffre hexadécimal à **4 bits**.

Méthode : regroupement

- **Binaire → octal (ou hexadécimal)** : on forme des groupes de 3 (ou 4) bits **à partir de la virgule**, vers la gauche pour la partie entière et vers la droite pour la partie fractionnaire, en complétant par des zéros aux extrémités ; on remplace chaque groupe par son chiffre.
- **Octal (ou hexadécimal) → binaire** : on remplace chaque chiffre par ses 3 (ou 4) bits.
- **Octal ↔ hexadécimal** : on passe par le binaire.

Exemple : regroupement

$(1101101{,}0111)_2$:

$$\text{hexadécimal : } \underbrace{0110}_{6}\ \underbrace{1101}_{D}\ ,\ \underbrace{0111}_{7} \Rightarrow (6D{,}7)_{16}$$

$$\text{octal : } \underbrace{001}_{1}\ \underbrace{101}_{5}\ \underbrace{101}_{5}\ ,\ \underbrace{011}_{3}\ \underbrace{100}_{4} \Rightarrow (155{,}34)_8$$

Vérification : $(6D{,}7)_{16} = 6\times 16 + 13 + \frac{7}{16} = 109{,}4375$ et $(155{,}34)_8 = 64 + 40 + 5 + \frac{3}{8} + \frac{4}{64} = 109{,}4375$.
Passage hexadécimal → octal : $(7F3)_{16} = 0111\,1111\,0011 = 011\,111\,110\,011 = (3763)_8$.

2.4 Conversion entre deux bases quelconques

On passe par le décimal (ou par le binaire quand les deux bases sont des puissances de 2).

Exemple : $(212)_3$ en base 5

$(212)_3 = 2\times 9 + 1\times 3 + 2 = 23$; puis $23 = 5\times 4 + 3$, donc $23 = (43)_5$.

3. Arithmétique binaire

Les règles sont celles du décimal, avec la base 2 : on retient une **retenue** dès que la somme d'une colonne atteint 2.

3.1 Addition

a	b	somme	retenue
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

avec retenue entrante : $1 + 1 + 1 = 11$ (somme 1, retenue 1).

Exemple : $1011 + 0110$

$$\begin{array}{rr} \text{retenues} & 1\,1\,1\,0 \\ & 1\,0\,1\,1 \\ + & 0\,1\,1\,0 \\ \hline & 1\,0\,0\,0\,1 \end{array} \qquad (11 + 6 = 17).$$

En hexadécimal, on retranche 16 et on retient 1 quand une colonne dépasse 15 : $(\texttt{A5})_{16} + (\texttt{3C})_{16}$: $5 + 12 = 17 = 16 + 1$ (chiffre 1, retenue 1), puis $10 + 3 + 1 = 14 = \texttt{E}$. Résultat : $(\texttt{E1})_{16} = 225$ $(165 + 60)$.

3.2 Soustraction

Règles : $0 - 0 = 0$, $1 - 0 = 1$, $1 - 1 = 0$ et $0 - 1 = 1$ avec un **emprunt** de 1 à la colonne suivante.

Exemple : $10110 - 01101$

$$\begin{array}{rr} \text{emprunts} & 1\,0\,0\,1 \\ & 1\,0\,1\,1\,0 \\ - & 0\,1\,1\,0\,1 \\ \hline & 0\,1\,0\,0\,1 \end{array} \qquad (22 - 13 = 9).$$

En pratique, les circuits n'effectuent pas de soustraction : ils additionnent le complément à 2 (section 4).

3.3 Multiplication et division

La table de multiplication se réduit à $0 \times 0 = 0 \times 1 = 1 \times 0 = 0$ et $1 \times 1 = 1$. On procède par produits partiels décalés, que l'on additionne.

Exemple : 1011×101

$$\begin{array}{rr} & 1\,0\,1\,1 \\ \times & 1\,0\,1 \\ \hline & 1\,0\,1\,1 \\ & 0\,0\,0\,0 \\ & 1\,0\,1\,1 \\ \hline & 1\,1\,0\,1\,1\,1 \end{array} \qquad (11 \times 5 = 55).$$

Décalages

Décaler un nombre binaire de k positions vers la **gauche** le multiplie par 2^k ; le décaler de k positions vers la **droite** le divise par 2^k (quotient entier). Exemple : $\texttt{1011} \ll 2 = \texttt{101100}$, soit $11 \times 4 = 44$.

La division s'effectue comme en décimal, par soustractions successives du diviseur. Pour $\texttt{1101} \div \texttt{11}$ $(13 \div 3)$: $\texttt{11} \leq \texttt{11}$ donne le bit de quotient 1 (reste 0) ; on abaisse le 0 : $\texttt{0} < \texttt{11}$, bit 0 ; on abaisse le 1 : $\texttt{1} < \texttt{11}$, bit 0. Quotient $(\texttt{100})_2 = 4$ et reste $(\texttt{1})_2 = 1$.

4. Représentation des entiers relatifs

Pour représenter des nombres négatifs sur n bits, on réserve le **bit de poids fort** au signe (0 pour $+$, 1 pour $-$). Trois conventions sont utilisées.

4.1 Signe et valeur absolue (SVA)

Les $n-1$ bits restants donnent la valeur absolue. Plage : $-(2^{n-1}-1)$ à $+(2^{n-1}-1)$. Le zéro a **deux** représentations ($+0$ et -0) et l'addition demande de comparer les signes : cette convention est peu adaptée au calcul.

4.2 Complément à 1 (CA1, ou complément restreint)

Le négatif d'un nombre s'obtient en **inversant tous ses bits** : $\mathrm{CA1}(N) = (2^n-1) - N$. Plage identique à la SVA, avec encore deux zéros (`0...0` et `1...1`). L'addition nécessite de rajouter la retenue finale au résultat.

4.3 Complément à 2 (CA2, ou complément vrai)

Définition : complément à 2

Le complément à 2 sur n bits d'un entier positif N est $\mathrm{CA2}(N) = 2^n - N = \mathrm{CA1}(N) + 1$. C'est la représentation de $-N$. La valeur d'un mot $a_{n-1}\dots a_0$ en CA2 est

$$N = -a_{n-1}\,2^{n-1} + \sum_{i=0}^{n-2} a_i\,2^i.$$

Obtenir le CA2 d'un nombre

- **Méthode 1** : inverser tous les bits, puis ajouter 1.
- **Méthode rapide** : recopier les bits de droite à gauche *jusqu'au premier* 1 *inclus*, puis inverser tous les bits restants.

Exemple : -25 sur 8 bits

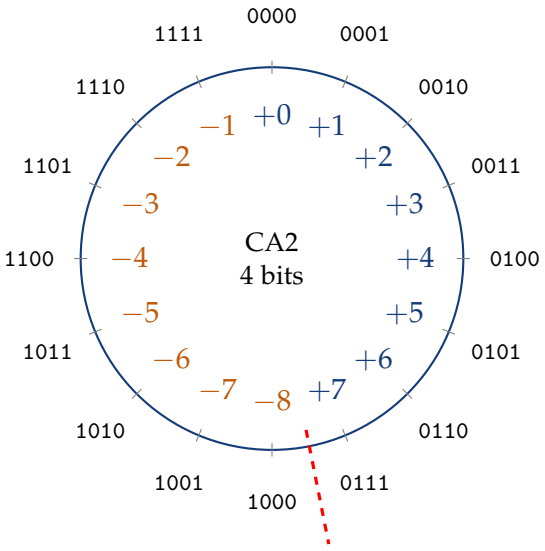
$25 =$ `00011001` ; inversion : `11100110` ; $+1$: `11100111`. Vérification : $256 - 25 = 231 =$ `11100111` et $-128+64+32+4+2+1 = -25$.
Méthode rapide sur `01101000` (104) : on recopie `1000` (jusqu'au premier 1 en partant de la droite), puis on inverse `0110` $\to$ `1001` : `10011000` $= 152 = 256 - 104$.

4.4 Comparaison des trois conventions

Représentation	Plage sur n bits	Zéros	Plage sur 8 bits
Non signé	$0 \dots 2^n-1$	1	$0 \dots 255$
SVA	$-(2^{n-1}-1) \dots 2^{n-1}-1$	2	$-127 \dots 127$
CA1	$-(2^{n-1}-1) \dots 2^{n-1}-1$	2	$-127 \dots 127$
CA2	$-2^{n-1} \dots 2^{n-1}-1$	1	$-128 \dots 127$

Les 16 mots de 4 bits et leur valeur selon la convention (N.S. : entier non signé) :

Code	N.S.	SVA	CA1	CA2	Code	N.S.	SVA	CA1	CA2
0000	0	+0	+0	0	1000	8	−0	−7	−8
0001	1	1	1	1	1001	9	−1	−6	−7
0010	2	2	2	2	1010	10	−2	−5	−6
0011	3	3	3	3	1011	11	−3	−4	−5
0100	4	4	4	4	1100	12	−4	−3	−4
0101	5	5	5	5	1101	13	−5	−2	−3
0110	6	6	6	6	1110	14	−6	−1	−2
0111	7	7	7	7	1111	15	−7	−0	−1



Le CA2 est la convention utilisée par les processeurs : le zéro est unique, la plage compte un négatif de plus que de positifs (asymétrie : -2^{n-1} n'a pas d'opposé représentable) et, surtout, le même additionneur sert pour les nombres signés et non signés.

Exemple : $+37$ et -37 sur 8 bits

$+37 = \texttt{00100101}$ dans les trois conventions. Pour -37 :

SVA	`1 0100101`	(bit de signe, puis 37)
CA1	`11011010`	(inversion de tous les bits)
CA2	`11011011`	(CA1 $+1$; on vérifie $256 - 37 = 219$)

4.5 Addition et soustraction en complément à 2

Arithmétique en CA2

L'arithmétique sur n bits est une arithmétique **modulo** 2^n. Comme $-b$ et $2^n - b$ sont identiques modulo 2^n :

$$a - b = a + \mathrm{CA2}(b) \pmod{2^n},$$

et la **retenue sortante** de l'addition est **ignorée**. La même opération sert donc à additionner et à soustraire.

Exemple : $13 - 6$ sur 5 bits

$13 = \texttt{01101}$, $6 = \texttt{00110}$, $\mathrm{CA2}(6) = \texttt{11010}$.

$$\texttt{01101} + \texttt{11010} = \underbrace{\texttt{1}}_{\text{ignorée}}\ \texttt{00111} \ \Rightarrow\ \texttt{00111} = 7.$$

Exemple : $37 - 58$ sur 8 bits

$-58 = \texttt{11000110}$ ($256 - 58 = 198$). Somme : $\texttt{00100101} + \texttt{11000110} = \texttt{11101011}$. Le bit de signe est 1 : le résultat est négatif ; son opposé est $\texttt{00010100} + 1 = \texttt{00010101} = 21$. Donc le résultat vaut -21, ce qui est bien $37 - 58$.

4.6 Dépassement de capacité (overflow)

Quand le résultat exact n'appartient pas à la plage représentable, le mot obtenu est faux : c'est un **dépassement**.

Détection d'un dépassement en CA2

- Additionner deux nombres de **signes opposés** ne provoque **jamais** de dépassement.
- Additionner deux positifs et trouver un résultat négatif, ou deux négatifs et trouver un résultat positif, signale un dépassement.
- Critère matériel : $V = c_{n-1} \oplus c_n$, où c_{n-1} est la retenue entrant dans le bit de signe et c_n la retenue sortante.

Exemple : dépassements sur 8 bits

- $100 + 50$: $\texttt{01100100} + \texttt{00110010} = \texttt{10010110}$, soit -106 au lieu de $150 > 127$. Retenue entrante 1, sortante 0 : $V = 1$, dépassement.
- $(-100) + (-50)$: $\texttt{10011100} + \texttt{11001110} = \underbrace{\texttt{1}}_{c_8}\texttt{01101010}$, soit $+106$ au lieu de $-150 < -128$. Retenue entrante 0, sortante 1 : $V = 1$, dépassement.

Extension de signe. Pour passer un entier CA2 de n à $n' > n$ bits sans changer sa valeur, on recopie le bit de signe à gauche : $\texttt{1011}$ (-5 sur 4 bits) devient $\texttt{11111011}$ (-5 sur 8 bits).

Représentation par excès (biaisée). On stocke $N + E$ avec un biais E (souvent 2^{n-1} ou $2^{n-1} - 1$) : le zéro est alors codé par E et l'ordre des nombres est conservé. Cette convention sert à coder l'exposant des nombres flottants (section 6).

5. Codes binaires

Un **code** associe une combinaison de bits à chaque élément d'information (chiffre, caractère, position). Un code est **pondéré** si chaque bit a un poids fixe (binaire naturel, DCB, Aiken), **continu** (ou adjacent) si deux mots consécutifs ne diffèrent que d'un bit (Gray), **auto-complémentaire** si le complément à 9 d'un chiffre s'obtient en inversant les bits.

5.1 Le code DCB (BCD)

Le **DCB** (décimal codé binaire, *BCD*) code **chaque chiffre décimal** sur un quartet en binaire naturel (poids 8 4 2 1). Les mots `1010` à `1111` sont interdits.

Exemple : DCB et binaire naturel

$(2026)_{10}$ = `0010 0000 0010 0110` en DCB, alors que $(2026)_{10}$ = `11111101010` en binaire naturel. Autre exemple : $(25)_{10}$ = `0010 0101` en DCB mais `11001` en binaire naturel.

Addition en DCB. On additionne les quartets comme des entiers binaires. Si la somme d'un quartet dépasse 9 (ou produit une retenue), le résultat n'est plus un chiffre DCB : on ajoute alors la **correction** `0110` **(+6)** à ce quartet, ce qui « saute » les six mots interdits.

Exemple : $47 + 38$ en DCB

	`0100`	`0111`	
$+$	`0011`	`1000`	
	`0111` `1111`	$(1111 > 9)$	$\Rightarrow$ 85.
$+$ correction		`0110`	
	`1000`	`0101`	

La correction sur le quartet de droite ($1111 + 0110 = 1\,0101$) produit une retenue qui passe au quartet de gauche : $0111 + 1 = 1000$.

5.2 Autres codes décimaux : Aiken et excès 3

Chiffre	0	1	2	3	4	5	6	7	8	9
DCB (8421)	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
Excès 3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100
Aiken (2421)	0000	0001	0010	0011	0100	1011	1100	1101	1110	1111

Le code **excès 3** s'obtient en ajoutant 3 au DCB ; il n'est pas pondéré. Les codes **excès 3** et **Aiken** sont **auto-complémentaires** : le complément à 9 d'un chiffre s'obtient en inversant les quatre bits (excès 3 : $3 = 0110 \to 1001 = 6 = 9 - 3$; Aiken : $2 = 0010 \to 1101 = 7 = 9 - 2$). Cela simplifie la soustraction en décimal codé.

5.3 Le code Gray (binaire réfléchi)

Dans le code Gray, **un seul bit change** entre deux mots consécutifs. On l'utilise dans les codeurs de position absolus et les tableaux de Karnaugh.

Déc.	Binaire	Gray	Déc.	Binaire	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Conversions binaire ↔ Gray

Pour un mot binaire $b_{n-1} \dots b_0$ et son équivalent Gray $g_{n-1} \dots g_0$:

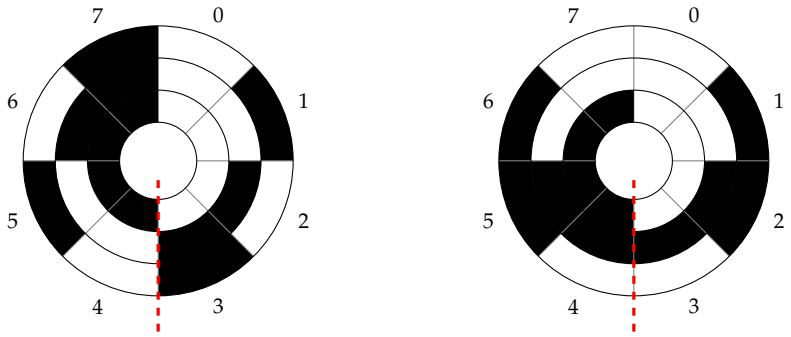
$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i \qquad\qquad b_{n-1} = g_{n-1}, \quad b_i = b_{i+1} \oplus g_i.$$

Autrement dit $G = B \oplus (B \gg 1)$. Le code s'obtient aussi par **réflexion** : le code à $n+1$ bits est le code à n bits précédé de 0, suivi du même code écrit en ordre inverse et précédé de 1.

Exemple : conversions

$(1011)_2 \to$ Gray : $g_3 = 1$, $g_2 = 1 \oplus 0 = 1$, $g_1 = 0 \oplus 1 = 1$, $g_0 = 1 \oplus 1 = 0$, soit 1110.
Retour : $b_3 = 1$, $b_2 = 1 \oplus 1 = 0$, $b_1 = 0 \oplus 1 = 1$, $b_0 = 1 \oplus 0 = 1$, soit 1011.

Pourquoi Gray pour un codeur de position ? Sur un disque codeur, chaque piste correspond à un bit ; en binaire naturel, le passage du secteur 3 (011) au secteur 4 (100) fait changer *trois* pistes à la fois. Si les capteurs ne commutent pas exactement en même temps, on peut lire transitoirement 000 ou 111, soit une erreur de plusieurs unités (jusqu'à 4). En Gray, le passage 010 → 110 ne change qu'une piste : l'erreur est au plus d'une unité.



Disques codeurs 3 bits (piste centrale : poids fort ; piste extérieure : poids faible ; noir = 1). Le trait rouge marque la frontière entre les secteurs 3 et 4.

5.4 Le code ASCII et Unicode

Le code **ASCII** associe à chaque caractère un mot de 7 bits (128 caractères). En pratique on le stocke sur un octet ; les codes 128 à 255 forment les jeux « étendus » (ISO 8859-1, etc.).

Codes (hexa)	Contenu	Exemples
00–1F et 7F	caractères de contrôle	NUL 00, tabulation 09, LF 0A, CR 0D, ESC 1B, DEL 7F
20	espace	
30–39	chiffres « 0 » à « 9 »	« 0 » = 30, « 9 » = 39
41–5A	majuscules	« A » = 41, « Z » = 5A
61–7A	minuscules	« a » = 61, « z » = 7A

Deux propriétés pratiques : la valeur d'un chiffre s'obtient en retranchant 30_{16} (ex. « 7 » = 37, donc 7) ; une minuscule s'obtient en ajoutant 20_{16} à la majuscule (un seul bit, le bit 5, change). Exemple : « ISSAT » = 49 53 53 41 54.

Unicode attribue un numéro à des dizaines de milliers de caractères (lettres accentuées, alphabet arabe, symboles). Son codage **UTF-8** utilise de 1 à 4 octets par caractère : les 128 premiers sont identiques à l'ASCII, « é » (U+00E9) s'écrit sur 2 octets (C3 A9), de même que les lettres arabes.

5.5 Codes détecteurs et correcteurs d'erreurs

Bit de parité. On ajoute un bit pour que le nombre total de 1 soit pair (**parité paire**) ou impair (**parité impaire**). Le récepteur recalcule la parité : elle détecte toute erreur portant sur un *nombre impair* de bits, mais ne la corrige pas et ne voit pas une double erreur.

Exemple : parité

« Y » = 1011001 contient quatre 1. Parité paire : bit ajouté 0, mot transmis 01011001. Parité impaire : bit ajouté 1, mot transmis 11011001.

Définition : distance de Hamming

La **distance de Hamming** $d(x, y)$ de deux mots de même longueur est le nombre de positions où ils diffèrent. Si d_{min} est la plus petite distance entre deux mots du code, le code **détecte** jusqu'à $d_{min} - 1$ erreurs et **corrige** jusqu'à $\left\lfloor \frac{d_{min}-1}{2} \right\rfloor$ erreurs.

Avec un bit de parité, $d_{min} = 2$: une erreur détectée, aucune corrigée.

Code de Hamming (7,4). Pour corriger une erreur simple, on ajoute 3 bits de contrôle p_1, p_2, p_3 à 4 bits de données $d_1 d_2 d_3 d_4$ ($d_{min} = 3$). Les 7 bits sont rangés dans l'ordre $p_1\, p_2\, d_1\, p_3\, d_2\, d_3\, d_4$ (les bits de contrôle occupent les positions 1, 2 et 4), avec

$$p_1 = d_1 \oplus d_2 \oplus d_4, \qquad p_2 = d_1 \oplus d_3 \oplus d_4, \qquad p_3 = d_2 \oplus d_3 \oplus d_4.$$

À la réception, on calcule le **syndrome** $(s_3 s_2 s_1)_2$ avec $s_1 = c_1 \oplus c_3 \oplus c_5 \oplus c_7$, $s_2 = c_2 \oplus c_3 \oplus c_6 \oplus c_7$, $s_3 = c_4 \oplus c_5 \oplus c_6 \oplus c_7$ (c_i : bit reçu en position i). S'il est nul, le mot est correct ; sinon il indique la **position du bit erroné**, qu'il suffit d'inverser.

Exemple : Hamming (7,4)

Données $d_1 d_2 d_3 d_4 = 1011$: $p_1 = 1 \oplus 0 \oplus 1 = 0$, $p_2 = 1 \oplus 1 \oplus 1 = 1$, $p_3 = 0 \oplus 1 \oplus 1 = 0$. Mot émis : 0110011.
Si le bit en position 5 est inversé, on reçoit 0110111 : $s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$, $s_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$, $s_3 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$. Syndrome 101 = 5 : on inverse le bit 5 et on retrouve 0110011.

6. Représentation des nombres réels

6.1 Virgule fixe

On réserve n_e bits à la partie entière et n_f bits à la partie fractionnaire (format noté $Q\,n_e.n_f$). La virgule n'est pas stockée : elle est *implicite*. La résolution est 2^{-n_f} et la plage 2^{n_e} (non signé).

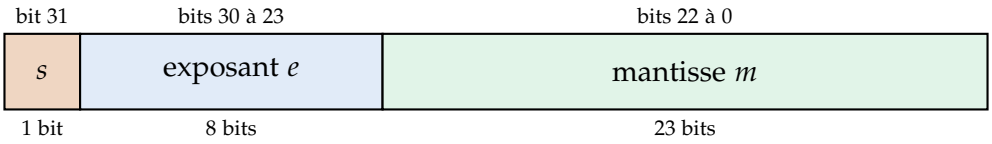
Exemple : format Q4.4 sur 8 bits

`0110.1010` $= 6 + \frac{1}{2} + \frac{1}{8} = 6{,}625$. Résolution : $2^{-4} = 0{,}0625$; plage non signée : 0 à 15,9375.

La virgule fixe est simple et rapide (arithmétique entière), mais sa plage est limitée : elle convient aux petits processeurs et aux traitements du signal en temps réel.

6.2 Virgule flottante : la norme IEEE 754

Un nombre est écrit $x = (-1)^s \times 1{,}m \times 2^{e-\text{biais}}$: un **signe** s, un **exposant** biaisé e et une **mantisse** (partie fractionnaire, le 1 initial étant implicite).



Format simple précision (32 bits).

	Simple précision	**Double précision**
Taille	32 bits	64 bits
Signe / exposant / mantisse	1 / 8 / 23	1 / 11 / 52
Biais	127	1023
Valeur (cas normal)	$(-1)^s\,1{,}m\,2^{e-127}$	$(-1)^s\,1{,}m\,2^{e-1023}$
Plage approximative	$\pm 3{,}4 \times 10^{38}$	$\pm 1{,}8 \times 10^{308}$
Précision décimale	≈ 7 chiffres	≈ 16 chiffres

Cas particuliers (simple précision). $e = 0$ et $m = 0$: zéro (signé) ; $e = 0$ et $m \neq 0$: nombres *dénormalisés* $(-1)^s\,0{,}m\,2^{-126}$; $e = 255$ et $m = 0$: $\pm\infty$; $e = 255$ et $m \neq 0$: NaN (*not a number*).

Codage d'un réel en simple précision

1. Le **signe** s vaut 0 si $x \geq 0$, 1 sinon. On travaille ensuite avec $|x|$.
2. Convertir $|x|$ en binaire (parties entière et fractionnaire).
3. **Normaliser** : écrire $|x| = 1, \ldots \times 2^E$ (décaler la virgule).
4. **Exposant biaisé** : $e = E + 127$, écrit sur 8 bits.
5. **Mantisse** : les bits après la virgule, complétés par des zéros à droite jusqu'à 23 bits.

Exemple : codage de 12,375

12,375 $=$ 1100,011 $=$ 1,100011 $\times 2^3$. Donc $s = 0$, $e = 3 + 127 = 130 =$ 10000010 et $m =$ 10001100…0. Le mot de 32 bits est

$$\underbrace{0}_{s}\ \underbrace{10000010}_{e}\ \underbrace{10001100000000000000}_{m}$$

soit, groupé par quartets : 0100 0001 0100 0110 0000 0000 0000 0000 = 0x41460000.

Exemple : codage de $-5{,}75$

5,75 $=$ 101,11 $=$ 1,0111 $\times 2^2$. Donc $s = 1$, $e = 2 + 127 = 129 =$ 10000001, $m =$ 0111 0…0 : le mot est 1100 0000 1011 1000 0…0 = 0xC0B80000.

Exemple : décodage de 0xC1480000

1100 0001 0100 1000 0…0 : $s = 1$; $e =$ 10000010 $= 130$, donc $E = 3$; $m =$ 1001…, soit $1{,}1001_2 = 1{,}5625$. Valeur : $-1{,}5625 \times 2^3 = -12{,}5$.

Remarque

Les réels flottants sont des **approximations**. $(0{,}1)_{10}$ n'ayant pas d'écriture binaire finie, il est arrondi à 0x3DCCCCCD $\approx 0{,}100000001490116$ en simple précision ; de même, en double précision, $0{,}1 + 0{,}2$ donne $0{,}30000000000000004$. On ne teste donc jamais l'égalité de deux flottants avec ==, mais l'écart avec une tolérance.

7. Synthèse du chapitre

À retenir

Valeur d'un nombre	$N = \sum a_i\, b^i$ (Horner : $((a_3 b + a_2)b + a_1)b + a_0$)
Décimal → base b	divisions successives (partie entière, restes de bas en haut) ; multiplications successives (partie fractionnaire, parties entières de haut en bas)
Bases $2, 8, 16$	groupes de 3 bits (octal) ou 4 bits (hexadécimal) à partir de la virgule
Capacité	n bits $\Rightarrow 2^n$ valeurs ; $n = \lceil \log_2 N \rceil$ pour N valeurs
CA2	$-N \to 2^n - N$: inverser puis $+1$; plage -2^{n-1} à $2^{n-1} - 1$
Soustraction	$a - b = a + \mathrm{CA2}(b)$, retenue finale ignorée
Dépassement	deux opérandes de même signe et résultat de signe opposé ; $V = c_{n-1} \oplus c_n$
DCB	un quartet par chiffre décimal ; correction $+6$ si quartet > 9 ou retenue
Gray	$G = B \oplus (B \gg 1)$; un seul bit change entre deux mots consécutifs
Parité / Hamming	parité : détecte 1 erreur ; Hamming (7,4) : corrige 1 erreur ($d_{min} = 3$)
IEEE 754 (32 bits)	$(-1)^s \times 1{,}m \times 2^{e-127}$; format $1 + 8 + 23$ bits

Erreurs fréquentes à éviter :

- lire les restes des divisions dans le mauvais sens (de haut en bas au lieu de bas en haut) ;
- oublier de compléter par des zéros les groupes de bits incomplets lors du regroupement ;
- appliquer les groupes de 4 bits à partir de la gauche pour la partie fractionnaire (il faut partir de la virgule) ;
- comparer des nombres en CA2 sans tenir compte du nombre de bits n choisi (le même mot n'a pas la même valeur sur 4 ou 8 bits) ;
- oublier que la retenue finale est ignorée en CA2, mais prise en compte dans le critère de dépassement ;
- confondre DCB et binaire naturel, ou oublier la correction $+6$;
- ajouter le biais 127 à l'exposant avant de normaliser, ou oublier le 1 implicite de la mantisse.

8. Exercices corrigés

Exercice 1 – Conversions

Effectuer les conversions suivantes :

1. $(1101101{,}011)_2$ en décimal ;
2. $(234{,}5)_{10}$ en binaire, en octal et en hexadécimal ;
3. $(7F3)_{16}$ en octal ;
4. $(212)_3$ en base 5.

Corrigé.

1. Partie entière : $64+32+8+4+1=109$. Partie fractionnaire : $\frac{1}{4}+\frac{1}{8}=0{,}375$. Résultat : 109,375.
2. $234=128+64+32+8+2=(11101010)_2$ et $0{,}5=(0{,}1)_2$, donc $(11101010{,}1)_2$. Regroupement par 3 : $011\,101\,010\,,100=(352{,}4)_8$. Regroupement par 4 : $1110\,1010\,,1000=(EA{,}8)_{16}$. Vérification : $14\times16+10+\frac{8}{16}=234{,}5$.
3. $7F3=0111\,1111\,0011=011\,111\,110\,011=(3763)_8$ (soit 2035 en décimal).
4. $(212)_3=23$ et $23=4\times5+3$, donc $(43)_5$.

Exercice 2 – Arithmétique binaire

Effectuer en binaire puis vérifier en décimal : (a) $101101+011011$; (b) $110010-001111$; (c) 1101×101.

Corrigé. (a) $45+27=72$: $101101+011011=1001000$ ($64+8=72$). (b) $50-15=35$: $110010-001111=100011$ ($32+2+1=35$). (c) $13\times5=65$: produits partiels 1101, 00000, 110100 ; somme $1101+110100=1000001$ ($64+1=65$).

Exercice 3 – Nombres signés sur 8 bits

1. Donner les représentations SVA, CA1 et CA2 de $+37$ et de -37.
2. Calculer $37-58$ en CA2.
3. Calculer $100+50$ en CA2 et conclure.
4. Quelle est la plage des entiers représentables en CA2 sur 12 bits ?

Corrigé.

1. $+37=00100101$ (identique dans les trois conventions) ; -37 : SVA 10100101, CA1 11011010, CA2 11011011.
2. $-58=11000110$; $00100101+11000110=11101011$ (pas de retenue sortante). Bit de signe 1 : négatif ; opposé : $00010101=21$. Résultat -21. Pas de dépassement (opérandes de signes opposés).
3. $01100100+00110010=10010110$, soit -106 : deux positifs donnent un négatif, il y a dépassement ($150>127$). Il faudrait au moins 9 bits.
4. $-2^{11}=-2048$ à $2^{11}-1=2047$.

Exercice 4 – Codes

1. Coder $(2026)_{10}$ en DCB et en excès 3.
2. Convertir 13 en code Gray sur 4 bits, puis décoder le mot Gray 0111.
3. Additionner $47+38$ en DCB.

4. Donner le mot ASCII de « Y » (0x59) avec un bit de parité paire en poids fort.

Corrigé.

1. DCB : 0010 0000 0010 0110.
   Excès 3 (on ajoute 3 à chaque chiffre : 5, 3, 5, 9) : 0101 0011 0101 1001.
2. $13 = 1101$: $g_3 = 1$, $g_2 = 1 \oplus 1 = 0$, $g_1 = 1 \oplus 0 = 1$, $g_0 = 0 \oplus 1 = 1$, soit 1011. Décodage de 0111 : $b_3 = 0$, $b_2 = 0 \oplus 1 = 1$, $b_1 = 1 \oplus 1 = 0$, $b_0 = 0 \oplus 1 = 1$, soit $0101 = 5$.
3. 0100 0111 + 0011 1000 = 0111 1111 ; le quartet de droite dépasse 9 : +0110 donne 1000 0101, soit 85.
4. 0x59 = 1011001 contient 4 uns (pair) : bit de parité 0, mot 01011001.

Exercice 5 – IEEE 754

1. Coder −5,75 en simple précision (donner le résultat en hexadécimal).
2. Décoder 0x41200000.
3. Coder 0,15625.

Corrigé.

1. 0xC0B80000 (voir l'exemple de la section 6).
2. 0100 0001 0010 0000 0 . . . 0 : $s = 0$, $e = 10000010 = 130$, $E = 3$, $m = 010\ldots$, soit 1,25. Valeur : $1{,}25 \times 8 = 10$.
3. $0{,}15625 = \frac{5}{32} = 0{,}00101 = 1{,}01 \times 2^{-3}$. Donc $e = -3 + 127 = 124 = 01111100$, $m = 01\,0\ldots0$: le mot est 0011 1110 0010 0000 0 . . . 0 = 0x3E200000.

9. Exercices proposés

Exercice A. Convertir $(1F{,}8)_{16}$ en décimal, puis $(0{,}3125)_{10}$ en binaire.
Réponses : 31,5 ; $(0{,}0101)_2$

Exercice B. Combien de bits faut-il pour représenter tous les entiers de 0 à 5000 ? Quel est le pas de quantification d'un CAN 10 bits de pleine échelle 3,3 V ?
Réponses : 13 *bits* ; $3{,}3/1024 \approx 3{,}2$ *mV*

Exercice C. Sur 6 bits en CA2, calculer $13 + 21$, $13 - 21$ et $(-13) + (-21)$; préciser les dépassements.
Réponses : 100010, *dépassement* ($34 > 31$) ; $111000 = -8$; *dépassement* ($-34 < -32$)

Exercice D. Coder 1975 en DCB. Décoder le mot DCB 0110 1000.
Réponses : 0001 1001 0111 0101 ; 68

Exercice E. Coder les données 1101 avec le code de Hamming (7,4).
Réponse : 1010101

Exercice F. Donner le mot Gray sur 5 bits de l'entier 22, puis l'entier dont le mot Gray est 11010.
Réponses : 11101 ; 19

— Fin du chapitre —